

SALESTORM — Design Pattern Mapping

1. Purpose

The SALESTORM architecture uses proven software design patterns to manage complexity across inventory reservation, payment processing, order lifecycle, external integrations, persistence, and failure recovery.

Patterns are selected based on actual problems in the system rather than being added only for theoretical compliance.

The primary goals are:

Reduce coupling between services and implementations

Support extensibility

Encapsulate complex workflows

Handle changing business rules

Protect external dependencies

Improve testability and maintainability

Provide clear ownership of domain behaviour

2. Design Pattern Summary

Design Pattern	SALESTORM Application	Primary Purpose
Strategy	Payment, pricing and delivery strategies	Select interchangeable business behaviour
Factory	Payment provider creation	Create appropriate provider implementations
State	Order and reservation lifecycle	Manage state-dependent behaviour
Observer	Order/payment events and notifications	Notify independent consumers
Adapter	Payment and delivery providers	Integrate external systems
Repository	Inventory, order and payment persistence	Abstract database operations
Facade	Checkout orchestration	Simplify complex workflows
Circuit Breaker	Payment gateway and external services	Prevent cascading failures
Transactional Outbox	Payment/order event publishing	Prevent event loss
Idempotent Consumer	Event processing	Prevent duplicate business effects

3. Strategy Pattern

Problem

SALESTORM may support multiple payment providers, pricing strategies or delivery methods.

Hard-coding provider-specific logic inside PaymentService would create strong coupling.

Solution

Define a common strategy interface and provide multiple implementations.

PaymentService

|
v

PaymentStrategy

|
+-----+
| | |
v v v

Razorpay Stripe MockPayment

Strategy Strategy Strategy

Example

```
interface PaymentStrategy {  
    initiatePayment()  
    checkStatus()  
    refund()  
}
```

Application

The same PaymentService can work with different payment providers without changing the core workflow.

Benefit

Easy provider replacement

Easy testing with mock providers

Supports future payment integrations

Reduces conditional logic

4. Factory Pattern

Problem

The system needs to create different payment-provider objects depending on configuration or business requirements.

Creating provider objects directly throughout the application creates unnecessary coupling.

Solution

Use a PaymentProviderFactory.

PaymentProviderFactory

```
|
+---- RazorpayProvider
|
+---- StripeProvider
|
+---- MockProvider
```

Example

```
createProvider("RAZORPAY")
```

↓

RazorpayProvider

Application

The Factory determines which concrete payment provider should be created while the rest of the system depends only on the provider abstraction.

Benefit

Centralized object creation

Easy addition of new providers

Reduced dependency on concrete classes

Works naturally with Strategy Pattern

5. State Pattern

Problem

Orders and inventory reservations move through multiple states.

Example:

CREATED

↓

PAYMENT_PENDING

↓

CONFIRMED

↓

PROCESSING

↓

SHIPPED

↓

OUT_FOR_DELIVERY

↓

DELIVERED

Reservations have their own lifecycle:

AVAILABLE



RESERVED



PAYMENT_PENDING



CONFIRMED



SOLD

Failure paths include:

PAYMENT_FAILED → RELEASED

TIMEOUT → RELEASED

Handling all these transitions using large if/else blocks becomes difficult to maintain.

Solution

Use the State Pattern to encapsulate state-specific behaviour and transitions.

Order

|

+-- CreatedState

+-- PaymentPendingState

+-- ConfirmedState

+-- ProcessingState

+-- ShippedState

+-- DeliveredState

Benefit

Explicit state transitions

Prevents invalid transitions

Easier lifecycle management

Easier addition of new states

Clear business rules

6. Observer Pattern

Problem

A successful order may trigger multiple independent actions:

Customer notification

Inventory update

Shipment processing

Analytics

Audit logging

The OrderService should not directly depend on every consumer.

Solution

Publish an event when an important business action occurs.

OrderService

|

| OrderConfirmed

↓

Message Broker

|
+-----→ Notification Service
|
+-----→ Shipment Service
|
+-----→ Analytics Service
|
+-----→ Audit Service

Example Events

OrderCreated

PaymentSucceeded

OrderConfirmed

ShipmentCreated

OrderDelivered

Benefit

Loose coupling

Independent consumers

Easy addition of new event consumers

Supports asynchronous processing

Improves scalability

7. Adapter Pattern

Problem

External payment and delivery providers may expose different APIs.

For example:

Provider A:

makePayment()

Provider B:

`createTransaction()`

Provider C:

`chargeCustomer()`

SALESTORM should not allow these differences to leak into business logic.

Solution

Use an Adapter for each external provider.

PaymentService



PaymentProvider Interface



+-----+

| Adapter Layer |

+-----+



Provider A Provider B

Application

The adapter converts the external provider's API into the internal SALESTORM interface.

Benefit

External API differences are isolated

Provider replacement becomes easier

Core business logic remains unchanged

Third-party dependencies are contained

8. Repository Pattern

Problem

Business services should not contain SQL/database-specific operations directly.

For example, `InventoryService` should not be responsible for constructing SQL queries.

Solution

Use repository abstractions.

`InventoryService`



`InventoryRepository`



`MySQLInventoryRepository`



SQL Database

Similarly:

`OrderService`



`OrderRepository`



`MySQLOrderRepository`

`PaymentService`



`PaymentRepository`



MySQLPaymentRepository

Benefit

Separates business logic from persistence

Easier unit testing

Database implementation can change independently

Centralizes data-access logic

9. Facade Pattern

Problem

Checkout involves several independent operations:

Validate Cart



Reserve Inventory



Create Payment



Confirm Payment



Create Order

Exposing all these internal services directly to the client would make the API complex.

Solution

Use a CheckoutFacade to provide a simplified entry point.

Customer



CheckoutFacade

|

+---- Cart Validation

|

+---- Inventory Reservation

|

+---- Payment

|

+---- Order Creation

Benefit

Simplifies client interaction

Hides internal workflow complexity

Provides a single orchestration boundary

Makes the purchase workflow easier to understand

10. Circuit Breaker Pattern

Problem

External payment gateways or delivery services may become slow or unavailable.

Without protection, repeated requests can:

increase latency

consume resources

create cascading failures

overload an already failing dependency

Solution

Use a Circuit Breaker around unstable external dependencies.

PaymentService

↓

Circuit Breaker



Payment Gateway

States

CLOSED

↓ failure threshold reached

OPEN

↓ recovery timeout

HALF_OPEN

↓ successful test

CLOSED

Behaviour

CLOSED: Requests flow normally.

OPEN: Requests fail fast without calling the unhealthy dependency.

HALF_OPEN: Limited requests test whether the dependency has recovered.

Benefit

Prevents cascading failures

Reduces unnecessary retries

Protects system resources

Improves recovery behaviour

11. Transactional Outbox Pattern

Problem

Consider this scenario:

Payment succeeds

↓

Order event needs to be published

↓

Order Service temporarily unavailable

If the event is only kept in memory, it could be lost.

Solution

Store the business change and event in the same database transaction.

Payment Transaction

|

+---- Payment Status = SUCCESS

|

+---- Outbox Event = PaymentSucceeded

Then:

Outbox

↓

Message Publisher

↓

Message Broker

↓

Order Service

Benefit

Prevents event loss

Supports reliable asynchronous processing

Enables retry after service recovery

Provides a durable recovery mechanism

12. Idempotent Consumer Pattern

Problem

Message brokers may deliver the same message more than once.

For example:

PaymentSucceeded

PaymentSucceeded

Processing both messages without protection could create duplicate orders.

Solution

The consumer maintains an idempotency key/event ID.

Event ID: EVT-1001

First delivery

→ Process

→ Store EVT-1001

Duplicate delivery

→ EVT-1001 already processed

→ Ignore / return existing result

Benefit

Prevents duplicate orders

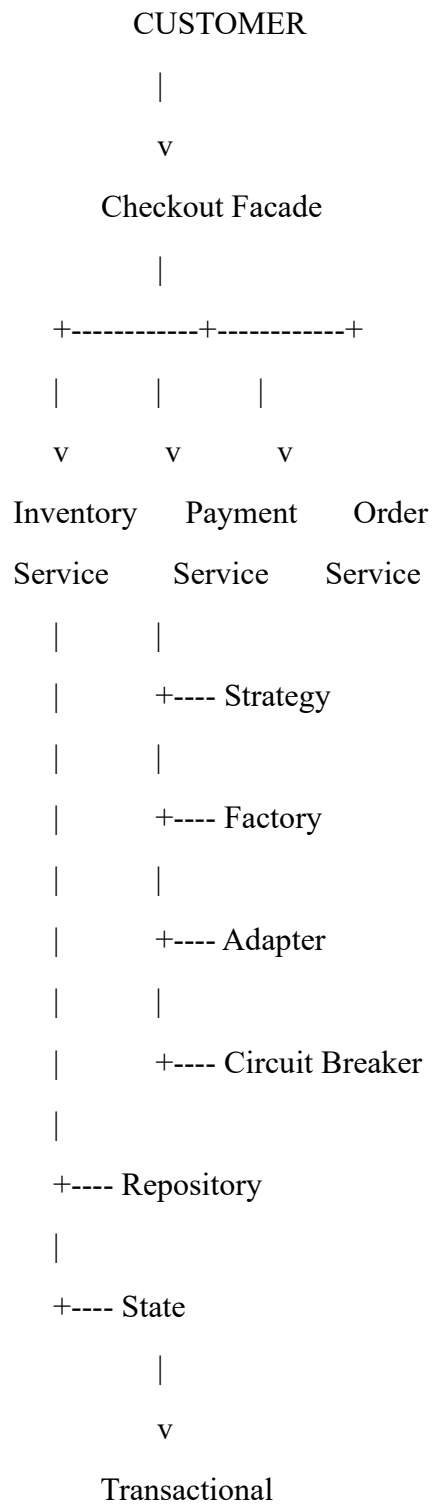
Makes message retries safe

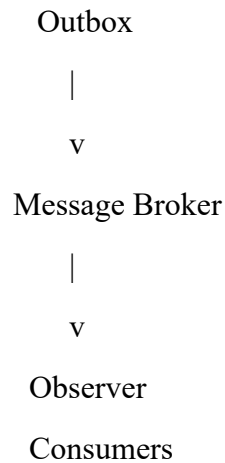
Supports at-least-once message delivery

Improves fault tolerance

13. Pattern Interaction in SALESTORM

The patterns work together rather than operating independently.





This provides:

Simple entry point → controlled business logic → reliable persistence → safe external integration → asynchronous recovery.

14. Pattern-to-Requirement Mapping

Business Requirement Pattern Used Why

Multiple payment providers Strategy + Factory Provider flexibility

Payment provider integration Adapter Isolate external APIs

Order lifecycle State Control valid transitions

Reservation lifecycle State Manage expiry/release

Customer notifications Observer Decouple event consumers

Database abstraction Repository Separate persistence

Complex checkout workflow Facade Simplify orchestration

Payment gateway failures Circuit Breaker Prevent cascading failures

Reliable event publishing Transactional Outbox Prevent event loss

Duplicate messages Idempotent Consumer Prevent duplicate effects

15. Design Pattern Trade-offs

Patterns introduce structure but also additional complexity.

Pattern Trade-off

Strategy	More classes/interfaces
Factory	Additional object-creation layer
State	More state classes
Observer	Event flow becomes harder to trace
Adapter	Additional abstraction layer
Repository	More code between service and database
Facade	Can become too large if responsibilities are not controlled
Circuit Breaker	Requires tuning thresholds and recovery behaviour
Transactional Outbox	Requires background publishing and cleanup
Idempotent Consumer	Requires tracking processed event IDs

Design Principle

SALESTORM uses patterns only where they solve a specific architectural problem. The objective is not maximum pattern usage, but controlled complexity and maintainability.

16. Final Design Pattern Mapping

The SALESTORM design combines structural, behavioural, and reliability-oriented patterns to create a scalable and fault-tolerant flash-sale platform.

The most critical patterns are:

For inventory correctness

State + Repository

For payment flexibility

Strategy + Factory + Adapter

For payment reliability

Circuit Breaker + Idempotency

For order lifecycle

State + Facade

For asynchronous communication

Observer + Transactional Outbox

For persistence

Repository

Together, these patterns support the core SALESTORM objectives:

10,000 concurrent purchase attempts → maximum 100 successful reservations → zero overselling → safe payment processing → recoverable order creation.